

Multi-Drone Agricultural Monitoring System

Theoretical Framework & Implementation Logic

Development Team

April 22, 2026

Contents

1	Introduction	3
2	Theoretical Background	3
2.1	Algorithm 3: Hybrid GOMWC + LERR + Shake	3
2.2	Drought Probability Model	3
2.3	Foundation Research	3
2.4	Sensor Fusion (Inverse-Variance Weighting)	4
2.5	Fault Detection Logic	4
2.6	Decentralized Swarm Ranging (INFOCOM 2021)	4
2.7	Energy-Constrained Coverage (ICRA 2024)	4
2.8	Cooperative Belief Space Planning (IROS 2024)	5
2.9	Centralized Communication Protocol	5
2.10	ROS Node Graph & Topics	5
2.11	Dynamic Coverage & 3D Flight	6
2.12	Adaptive Waypoint Grids	6
3	Frequently Asked Questions (FAQ)	7
3.1	Probabilty & Data Merging	7
3.2	Drone Behavior & Allocation	7
4	Implementation Analysis	8
4.1	Expected vs. Actual Implementation	8
4.2	Key Code Implementations	9
4.2.1	Sensor Fusion Logic	9
4.2.2	Fault Detection Threshold	9
5	File Structure & Component Overview	9
5.1	Directory Hierarchy	9
6	Codebase Architecture & Logic	10
6.1	DroughtCast Implementation (Brust et al., 2021)	11
6.2	Risk-Aware Allocator (Logic Layer)	11
6.3	Main Controller (ROS Integration)	11
6.4	Swarm Ranging (INFOCOM 2021)	12
6.5	Cooperative Belief Space (IROS 2024)	12
6.6	Energy-Constrained Coverage (ICRA 2024)	12
6.7	Hardware Failure Simulation & Reserve Deployment	13
6.8	Surveillance Stop Condition	13
7	System Parameters	14
7.1	Current Limitations	14
7.2	Research Modules Implemented	14
8	Implementation FAQs	14
8.1	Future Roadmap	15
9	Recent Changes & Bug Fixes	16
9.1	Session: 5 April 2026 — Reserve Drone Activation & Stop-Condition Fixes . . .	16
9.2	Commit 69c7b75 — 3 April 2026: UGV Deadlocks, Crash Visuals, Dead-Drone Edge Cases	17
9.3	Commit f6431ef — 2 April 2026: Fleet Stagnation & Dynamic Reconfiguration	17

9.4	Commit 8ed9f43 — 13 March 2026: Scanning State Machine Fix	18
-----	--	----

1 Introduction

This document outlines the theoretical foundations and decision-making logic behind the Multi-Drone Agricultural Monitoring System. The system deploys a fleet of **11 autonomous drones** (9 active explorers + 2 reserve drones) over a **single unified 21.5 m × 21.5 m workspace** using **Algorithm 3 (Hybrid GOMWC + LERR + Shake)** with **dynamic Voronoi partitioning**, monitors farmland for drought risk using meteorological data, and validates data integrity through multi-sensor fusion. The system also simulates two hardware failures at randomised coverage thresholds and automatically deploys reserve drones via the Central Tower’s heartbeat monitor.

2 Theoretical Background

2.1 Algorithm 3: Hybrid GOMWC + LERR + Shake

The system implements a robust coverage algorithm designed for multi-agent coordination:

- **GOMWC (Greedy Overlap-Minimizing Waypoint Coverage):** Drones prioritize the closest unvisited waypoint to maximize immediate coverage gain.
- **LERR (Local Energy Risk Reduction):** To prevent collisions and clustering, drones apply a "Proactive Repulsion" penalty. If a drone selects a target, that target becomes "expensive" (high cost) for all other drones, forcing them to diverge.
- **Shake Mechanism:** A stochastic deadlock resolution strategy. If the swarm’s coverage progress stagnates (e.g., local minima), a "Shake" event forces a random bounded jump to a new target, acting as a simulated annealing step.

2.2 Drought Probability Model

The system estimates drought risk based on the methodology proposed in *"DroughtCast: A Machine Learning Forecast of the United States Drought Monitor"* (Brust et al., 2021).

2.3 Foundation Research

The project builds upon the swarm architecture detailed in *"Multi-Robot Communication-Aware Cooperative Belief Space Planning with Inconsistent Beliefs: An Action-Consistent Approach"* (Kundu et al., IROS 2024).

Rather than relying on single inputs, the model aggregates multiple meteorological indices:

- **SPI (Standardized Precipitation Index):** Measures rainfall deficit.
- **SMI (Soil Moisture Index):** Assesses water availability in the root zone.
- **VCI (Vegetation Condition Index):** Evaluates crop health via satellite imagery proxies.
- **TCI (Temperature Condition Index):** Accounts for thermal stress.

The final probability $P_{drought}$ is derived from a weighted non-linear combination of these features, normalized to a $[0, 1]$ range using a logistic function:

$$P(y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \dots + \beta_n X_n)}} \quad (1)$$

2.4 Sensor Fusion (Inverse-Variance Weighting)

To mitigate sensor noise and errors, the system employs **Inverse-Variance Weighting** for merging probabilities from multiple drones. This method gives more weight to precise sensors (low variance) and less weight to noisy ones.

Given a set of measurements x_i with known variances σ_i^2 , the fused estimate $\hat{x}$ and its variance $\hat{\sigma}^2$ are calculated as:

$$w_i = \frac{1}{\sigma_i^2} \quad (2)$$

$$\hat{x} = \frac{\sum_i w_i x_i}{\sum_i w_i} \quad (3)$$

$$\hat{\sigma}^2 = \frac{1}{\sum_i w_i} \quad (4)$$

This ensures that the final fused probability is statistically the most reliable estimate possible given the available data.

2.5 Fault Detection Logic

The system automatically detects malfunctioning sensors using statistical hypothesis testing. A sensor reading x_{sensor} is flagged as **faulty** if it deviates significantly from the expected model prediction x_{model} or the consensus of other drones.

The fault condition is triggered if:

$$|x_{sensor} - x_{model}| > (2 \cdot \sigma_{noise} + \delta_{threshold}) \quad (5)$$

Where:

- σ_{noise} is the standard deviation of the sensor (noise floor).
- $\delta_{threshold}$ is a configurable safety margin (e.g., 15%).

If a fault is detected, the system automatically dispatches an **Auditor Drone** from the reserve pool to verify the reading.

2.6 Decentralized Swarm Ranging (INFOCOM 2021)

The implemented localization system operates on the principle of **Relative Positioning** via Ultra-Wideband (UWB) ranging. Unlike GPS, which provides absolute coordinates, UWB modules measure the Time-Of-Flight (ToF) between transceiver pairs to determine distance d_{ij} between Drone i and Drone j . The system resolves the relative swarm geometry using **Multilateration**:

$$\min_{x,y} \sum_{i=1}^N (\sqrt{(x - x_i)^2 + (y - y_i)^2} - d_{measured})^2 \quad (6)$$

This optimization problem (Least-Squares) allows the swarm to maintain formation and collision avoidance even in GPS-denied environments.

2.7 Energy-Constrained Coverage (ICRA 2024)

The coverage problem is modeled as a variation of the **Vehicle Routing Problem (VRP)** with energy constraints. The classic coverage objective is to visit all target nodes V such that the total path length is minimized. However, our implementation introduces a battery constraint, $B(t) > B_{critical}$. The system solves this by introducing a mobile recharging station (UGV). The planning logic transforms into a **Rendezvous Problem**: finding the optimal intersection point P_{meet} and time t_{meet} where the drone and UGV paths intersect before $B(t) \rightarrow 0$.

2.8 Cooperative Belief Space Planning (IROS 2024)

While the current implementation simplifies the full belief space planning, the theoretical foundation relies on reducing state uncertainty Σ . In a cooperative belief space, the uncertainty of the entire swarm state X is coupled. Effective planning minimizes the trace of the covariance matrix:

$$J = \text{tr}(\Sigma_{\text{swarm}}) \quad (7)$$

The system now explicitly computes this metric using the **Moore-Penrose Pseudo-Inverse** to handle potential rank-deficient anchor configurations (e.g., collinearity):

$$\text{tr}(\Sigma) \approx \text{tr}((J^T J)^+ \cdot \text{MSE}) \quad (8)$$

This robust formulation allows the `swarm_localization` node to provide a live "Belief Uncertainty" score without numerical instability. Our active "Auditor" drone recruitment is a direct application of this principle: deploying an agent specifically to reduce the uncertainty (variance) of a high-risk belief state.

2.9 Centralized Communication Protocol

To ensure robust fleet coordination, we implemented a centralized architecture managed by a static "Central Tower" node. The system enforces a strict **3-Way Handshake** to establish reliable links with all 11 drones and 2 UGVs:

1. **Tower Broadcast (SYN)**: The Central Tower broadcasts a `HELLO` message on the global topic.
2. **Agent Response (SYN-ACK)**: Each agent (Drone/UGV) detects the broadcast and replies with an `AGENT_HI_<ID>` message.
3. **Async Acknowledgement (ACK)**: The Tower places incoming requests into a FIFO processing queue. A separate 10Hz timer service processes the queue, broadcasting a `TOWER_ACK_<ID>` confirmation. This asynchronous decoupling simulates realistic server load and latency.

This ensures that no agent is left orphaned and provides visual confirmation of the fleet's connectivity status in the system logs.

2.10 ROS Node Graph & Topics

The simulation architecture is built upon the Robot Operating System (ROS) Noetic. The primary data flow relies on the following interactions:

Topic Name	Message Type	Description
<code>/central/comm</code>	<code>std_msgs/String</code>	Downlink from Tower to Fleet (Broadcasts)
<code>/comm/agents</code>	<code>std_msgs/String</code>	Uplink from Agents to Tower (Responses)
<code>/drone_{id}/cmd_vel</code>	<code>geometry_msgs/Twist</code>	Velocity commands for flight control
<code>/drone_{id}/odom</code>	<code>nav_msgs/Odometry</code>	Position and velocity feedback
<code>/drone_{id}/battery</code>	<code>std_msgs/Float32</code>	Simulated battery level (0-100%)

Table 1: Key ROS Topics and Interface Definitions

The **Central Agent** node acts as the graph sink for fleet status messages, while the **Area Explorer** nodes (one per drone) act as independent agents that fuse local odometry with global commands.

2.11 Dynamic Coverage & 3D Flight

Unlike simple waypoint counting, the system implements a ****Swept Volume Coverage**** metric.

- **Dynamic Grid Painting:** As a drone moves, it updates a probabilistic occupancy grid map of the farmland. A cell is marked as "Covered" only if it falls within the drone's sensor cone.
- **Altitude-Dependent FOV:** The sensor footprint is modeled perfectly relative to the visual cone geometry. The ground radius R_{fov} scales linearly with altitude h : $R_{fov} = 0.625 \cdot h$. This ensures a 1:1 match between the logical area claimed covered and the actual graphical footprint seen in simulation.
- **Variable Altitudes & Stability:** Drones autonomously select staggered flight layers between 2.0m and 3.0m to enhance vertical deconfliction and avoid physical downwash. To eliminate altitude drift (v_z) and gravity-induced simulation crashes, the models employ a zero-gravity planar simulation model ($< gravity > 0 < /gravity >$ in Gazebo), keeping them locked exactly at their spawned altitude layers permanently.

2.12 Adaptive Waypoint Grids

To ensure rigorous sensing resolution across different farm sizes, the waypoint grid density is calculated via a predefined discrete 1.5m matrix overlaid on the farm bounding box:

- **1.5m Matrix Spacing:** Target waypoints are generated explicitly every 1.5 meters.
- **Circular Boundary Filtering:** Nodes mathematically outside the predefined crop area radius are stripped from the target list computationally. The algorithmic state expects 100% of these internal waypoints to be visited before the area is marked fully explored.
- **12m Areas (e.g., Areas 2, 5):** $\frac{12m}{1.5m} \approx 8 \text{ gaps} \rightarrow 9 \times 9 = \mathbf{81 \text{ Waypoints}}$.
- **10m Areas (e.g., Areas 1, 3, 4):** $\frac{10m}{1.5m} \approx 6.6 \text{ gaps} \rightarrow 7 \times 7 = \mathbf{49 \text{ Waypoints}}$.

This explains why larger areas have significantly more waypoints (81 vs 49), ensuring the physical sampling density reliably traces the exact perimeters.

3 Frequently Asked Questions (FAQ)

3.1 Probabilty & Data Merging

Q: How do we merge probabilities from multiple drones?

A: We use a technique called **Sensor Fusion** (specifically, Inverse-Variance Weighting). Imagine Drone A reports a drought risk of 60% with high confidence (low noise), and Drone B reports 40% with low confidence (high noise). Instead of just averaging them to 50%, the system "trusts" Drone A more. The mathematical formula (see Section 2.2) ensures that the final combined probability leans closer to the more accurate sensor.

Q: What happens if the Model and the Sensor disagree?

A: The system trusts the drone's physical sensor reading *unless* the deviation is statistically impossible (see Fault Detection).

- **Small Disagreement:** The system fuses the model prediction and sensor reading mathematically to find a middle ground.
- **Large Disagreement:** The system flags the sensor as potentially faulty and dispatches a second drone (Auditor) to double-check.

3.2 Drone Behavior & Allocation

Q: How are drones assigned to areas?

A: Assignment is based on **Risk Prioritization**. 1. **High Risk Areas (> 70%):** Get multiple drones (up to 3) to ensure frequent monitoring. 2. **Medium Risk Areas (40–70%):** Get at least 1 drone. 3. **Low Risk Areas:** Monitored if resources allow.

We also keep a **Reserve Pool** (approx. 10% of the fleet) idle at the base. These reserves are only launched if a sensor failure is detected and an Auditor is needed immediately.

Q: Why do drones sometimes hover instead of moving?

If a drone drifts near the boundary of its assigned circular area, it enters a "Hold" or "Correction" state. It stops exploring and applies a velocity vector towards the center of the area to ensure it doesn't fly into restricted airspace or crash into a neighboring sector.

Q: Why does the simulation close automatically?

A: To facilitate batch testing and Monte Carlo experiments, the `area_explorer` node is configured to trigger an **Auto-Shutdown** sequence 5 seconds after all exploration missions are reported as complete.

4 Implementation Analysis

4.1 Expected vs. Actual Implementation

The initial design specifications (outlined in `expected_work.md` and the reference paper *Drought-Cast*) proposed an end-to-end ML pipeline. Below is a comparison of the theoretical design versus the current simulated implementation.

Feature	Expected Design	Current Implementation
Drought Prediction	LSTM Neural Network trained on historical Kaggle meteorological data (SPI, SMI, VCI).	LSTM Integrated: The system runs a trained PyTorch LSTM model sequence-to-one inference inside the ROS loop (<code>drought_probability_model.py</code>).
Data Sources	Real-time satellite imagery (MODIS) and weather station APIs.	Historical Data & Portable Inference: Ingests CSV time-series data locally. If omitted (to save repo size), inference gracefully cascades to a lightweight, pre-extracted 90-day sequence binary (<code>LSTM/reference_sequence.npy</code>).
Coverage Optimization	<code>scipy.optimize</code> for perfect non-overlapping circles.	Algorithm 3 + Voronoi: Hybrid GOMWC + LERR + Shake on a unified 21.5 m $\times$ 21.5 m workspace with dynamic Voronoi partitioning (<code>algo3_sim.py</code>).
Fleet Composition	10 active + 1 reserve.	9 active explorers + 2 reserve drones. Reserves hold position and are promoted on hardware failure.
Fault Detection	Statistical hypothesis testing (2σ deviation).	Implemented exactly as designed. Compares sensor readings against the model's baseline prediction.
Swarm Ranging	Full centralized graph optimization for all N drones.	Decentralized Multilateration: Implemented a lightweight Least-Squares solver per drone (<code>swarm_localization.py</code>) consuming simulated UWB ranges.
Energy Planning	Complex VRP with Time Windows (VRPTW) and charging constraints.	Multi-UGV Cooperative Rendezvous: Two UGVs independently run Dijkstra paths to critical drones; battery reset to 100% after 1 s dwell (<code>energy_planner.py</code>).

Table 2: Design vs. Implementation Comparison

4.2 Key Code Implementations

The core logic for our ****Sensor Fusion**** and ****Fault Detection**** is critical to the system's reliability. Below are the specific functions driving these behaviors.

4.2.1 Sensor Fusion Logic

We use **Inverse-Variance Weighting** to merge conflicting probability estimates. This ensures that a drone with a noisy sensor (high variance) has less influence on the final decision than a drone with a precise sensor.

```
1 def fuse_probabilities(prob_a, var_a, prob_b, var_b):
2     """
3     Merges two probability estimates using inverse-variance weighting.
4     Formula: (w1*x1 + w2*x2) / (w1 + w2) where w = 1/variance
5     """
6     weight_a = 1.0 / var_a
7     weight_b = 1.0 / var_b
8
9     fused_prob = (weight_a * prob_a + weight_b * prob_b) / (weight_a + weight_b)
10
11     fused_var = 1.0 / (weight_a + weight_b)
12
13     return fused_prob, fused_var
```

Listing 1: Inverse-Variance Fusion Logic

4.2.2 Fault Detection Threshold

The system flags a sensor as "Faulty" if its reading deviates from the expected model prediction by more than dynamic threshold (driven by the sensor's known noise profile).

```
1 def detect_fault(sensor_val, model_val, sensor_noise_std):
2     # Threshold is 2 * standard_deviation (95% confidence interval)
3     # plus a small base margin (0.15)
4     dynamic_threshold = (2 * sensor_noise_std) + 0.15
5
6     deviation = abs(sensor_val - model_val)
7
8     if deviation > dynamic_threshold:
9         return True # FAULT DETECTED
10
11     return False
```

Listing 2: Adaptive Fault Detection

5 File Structure & Component Overview

This section provides a detailed breakdown of the codebase organization, highlighting the purpose of key directories and files.

5.1 Directory Hierarchy

- **scripts/**: Contains the core Python ROS nodes.
 - **algo3_sim.py**: Hybrid GOMWC + LERR + Shake algorithm with dynamic Voronoi partitioning.
 - **area_allocation.py**: Legacy risk-based drone allocation (research reference).
 - **area_explorer.py**: Main mission controller and ROS node entry point. Houses DroneExplorer, BackupDrone, AreaCoverageController, Battery, and the main() loop.
 - **central_agent.py**: Central Tower node. Publishes HELLO broadcasts, processes the ACK queue, tracks heartbeats, and fires DEPLOY_RESERVE on drone death.

- `drone_comm.py`: Virtual drone communications relay. Implements the `VirtualDrone` class; silences dead drones via `/swarm/drone_death` callback.
- `drone_controller.py`: Low-level PID flight controller.
- `drought_probability_model.py`: LSTM inference wrapper.
- `energy_planner.py`: Multi-UGV energy-aware path planner. Implements Dijkstra on a 2 m grid and coordinates two independent UGV charging missions.
- `sensor_fault_detection.py`: Statistical fault detection (2σ hypothesis test).
- `spawn_fleet.py`: Gazebo SDF spawner. Places drones in a 16 m radius circle; reserve drones are separated from explorers.
- `swarm_localization.py`: Decentralised UWB-based positioning (INFOCOM 2021).
- `ugv_comm.py`: UGV communications relay (mirrors `drone_comm.py` for UGVs).
- `ugv_manager.py`: Mobile UGV physics, patrol logic, and proximity charging.
- `uwb_simulator.py`: Simulates UWB hardware ranges.
- **launch/**: ROS launch files for system startup.
 - `explore_areas.launch`: The master entry point. Launches the simulation, spawns drones, and starts the explorer node.
 - `spawn_drones.launch`: Helper script to spawn 11 quadcopters in Gazebo with specific namespaces (`drone_0` to `drone_10`).
- **config/**: Configuration files.
 - `areas.yaml`: Defines the 10 monitored farmland areas (coordinates, crop type, historical drought data).
- **LSTM/**: Deep Learning resources.
 - `model.py`: PyTorch definition of the `DroughtLSTM` class (Input: 6 features, Hidden: 64 units).
 - `lstm_model.pth`: Pre-trained binary weights for the model.
- **worlds/**: Simulation environments.
 - `field_areas.world`: A Gazebo world file pre-configured with the farmland terrain.
- **models/**: 3D assets.
 - `quadcopter/`: SDF model description for the Iris drone used in the simulation.

6 Codebase Architecture & Logic

This section details the functional implementation of the active nodes, mapping specific classes and functions to the theoretical research papers.

6.1 DroughtCast Implementation (Brust et al., 2021)

Core Logic: `drought_probability_model.py`

The system implements the multi-index probabilistic model proposed in *DroughtCast* using a Long Short-Term Memory (LSTM) network.

- **Class:** `DroughtLSTM` (in `LSTM/model.py`)

- **Role:** The neural architecture. Defined as a 2-layer LSTM with 64 hidden units, followed by a fully connected layer and Sigmoid activation.
- **Why?:** Captures temporal dependencies in meteorological time-series data (90-day look-back).
- **Functions:** `predict_from_csv()` and `predict_from_reference_sequence()`
- **How it works:**
 1. Attempts to load 90 days of daily stats from the massive raw CSV repository.
 2. If the CSV is omitted for portability, automatically loads `reference_sequence.npy` (a lightweight Numpy binary array representing a valid 90-day feature slice).
 3. Normalizes data to $[0, 1]$.
 4. Feeds tensor of shape $(1, 90, 6)$ into the model.
 5. Returns $P(drought) \in [0, 1]$.

6.2 Risk-Aware Allocator (Logic Layer)

Core Logic: `area_allocation.py`

This module translates the abstract probabilities into concrete drone assignments.

- **Class:** `DynamicDroneAllocator`
- **Function:** `allocate_drones(areas, drones)`
- **Mechanism:**
 - **Sorting:** `areas.sort(key=lambda x: x.drought_probability, reverse=True)`.
 - **Phase 1 (Baseline):** `assign_drone(area_id)` for all areas to ensure minimal coverage.
 - **Phase 3 (High-Risk Injection):** For areas with $P > 0.7$, iterate and assign additional drones up to `max_drones_per_area`.
 - **Reserves:** Uses `reserve_percentage` to keep a portion of the fleet idle for fault-recovery (Auditor missions).

6.3 Main Controller (ROS Integration)

Core Logic: `area_explorer.py`

The `area_explorer` node binds the logic together into a ROS application.

- **Class:** `DroneExplorer`
- **Role:** Manages the state machine for a single drone (Init $\rightarrow$ Explore $\rightarrow$ Return).
- **Function:** `explore()`
- **Key Implementation:**
 - **Path Planning:** Generates "Lawnmower" grid waypoints inside `is_within_area_bounds()`.
 - **Flight Control:** Implements a P-Controller.
 - **Critical Fix:** The raw velocity vector (v_x, v_y) is rotated by the drone's Yaw ψ to ensure correct movement relative to the drone's heading:

$$R(\psi) = \begin{bmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{bmatrix} \quad (9)$$

6.4 Swarm Ranging (INFOCOM 2021)

Core Logic: `swarm_localization.py`

Implements the "Range-Based Relative Positioning" protocol.

- **Class:** `SwarmLocalization`
- **How:** Subscribes to `/swarm/uwb_ranges` (mocked hardware).
- **Function:** `compute_position()`
- **Algorithm:** Non-linear Least Squares.

$$\text{minimize } \sum (d_{measured} - ||P_{est} - P_{anchor}||)^2 \quad (10)$$

- **Role:** Allows drones to know where they are relative to neighbors without GPS.

6.5 Cooperative Belief Space (IROS 2024)

Core Logic: Integrated into `swarm_localization.py`

- **Feature:** Real-time uncertainty quantification.
- **Implementation:** After solving for position P , we calculate the Jacobian J .
- **Metric:** Trace of the Covariance Matrix ($\text{tr}(\Sigma)$), where $\Sigma \approx (J^T J)^{-1}$.
- **Role:** Provides a "Confidence Score" for the location estimate, used to trigger auditor drones if uncertainty is too high.

6.6 Energy-Constrained Coverage (ICRA 2024)

Core Logic: `energy_planner.py`

- **Class:** `EnergyAwarePlanner`
- **Role:** Fleet resource manager for two UGVs (`ugv1`, `ugv2`).
- **How:**
 1. **Monitoring:** `bat_cb()` and `odom_cb()` keep per-drone battery and position state.
 2. **Geofence:** Drones that are at spawn ($y < -15.0$ m) are excluded from UGV dispatch.
 3. **Priority Queue:** The planner continuously evaluates a dynamic priority score for each drone:
$$J_{priority} = \frac{(100 - \text{BatteryLevel})^2}{\text{DistanceToUGV} + 1} \quad (11)$$
This non-linear urgency factor ensures that critical drones ($< 20\%$) are prioritised over closer but healthier drones.
 4. **Multi-UGV Assignment:** Each IDLE UGV independently selects the highest-scoring *un-targeted* drone. A shared `targeted_drones` set prevents both UGVs from chasing the same drone simultaneously.
 5. **Global Path Planning:** The UGV utilises **Dijkstra's Algorithm** on a 2 m grid (50×50 cells, covering a $100 \text{ m} \times 100 \text{ m}$ world) to calculate the shortest collision-free path.

6. **Navigation:** The UGV follows the Dijkstra path waypoints via P-control at up to 2.0 m/s, falling back to direct P-control if Dijkstra returns an empty path.
7. **Proximity Charging:** When the UGV is within 2.0 m of the target:
 - It holds position for 1.0 s (simulating connection).
 - It publishes `Float32(100.0)` to `/drone_{id}/charge_cmd`, resetting battery to 100%.
 - The UGV returns to `IDLE` and is immediately available for the next assignment.

6.7 Hardware Failure Simulation & Reserve Deployment

Core Logic: `area_explorer.py` (main loop) & `central_agent.py`

The system injects two simulated hardware failures per run at randomised coverage thresholds:

- **Failure 1:** Triggered at coverage $\in \mathcal{U}(10\%, 25\%)$.
- **Failure 2:** Triggered at coverage $\in \mathcal{U}(50\%, 70\%)$.

On each failure:

1. Drone teleported to $z = -100$ m (clears physics handle); static `dead_quadcopter` SDF spawned at crash location.
2. RViz FOV cone `Marker` deleted (`action=DELETE`).
3. `Float32(100.0)` published to `/drone_{id}/battery` (prevents UGV dispatch to wreck).
4. `Int32(drone_id)` published to `/swarm/drone_death`; `drone_comm.py` silences heartbeat.
5. Central Tower timeout (> 15 s) triggers `DEPLOY_RESERVE` command.
6. `BackupDrone.deployed = True` terminates the hold-position thread; new `DroneExplorer` created; Voronoi re-partitioned.

6.8 Surveillance Stop Condition

Core Logic: `area_explorer.py` main loop, `algo3_sim.py` `step()`

When total field coverage $\geq 99.9\%$, `step()` returns "Coverage Complete" (grid is **not** reset). The main loop then:

1. Publishes `Twist()` to every explorer `/drone_{id}/cmd_vel`.
2. Publishes `Bool(True)` on `/mission_complete` — parks UGVs and stops Central Tower.
3. Breaks, generates reports, then `rospy.signal_shutdown()` after 5 s.

7 System Parameters

Parameter & Value
Total drones & 11
Active explorers & 9
Reserve drones & 2
Unified workspace & 21.5 m $\times$ 21.5 m
Spawn ring radius & 16 m (centre at origin)
Waypoint grid spacing & 1.5 m
Battery capacity & 1500 mAh
Low-battery threshold & 30% (UGV dispatch)
Critical-battery threshold & 20% (drone returns)
UGV charging dwell & 1 s (then instant 100% reset)
UGV Dijkstra grid & 2 m resolution, 50 $\times$ 50 cells
Failure 1 trigger & $\mathcal{U}(10\%, 25\%)$ coverage
Failure 2 trigger & $\mathcal{U}(50\%, 70\%)$ coverage
Heartbeat timeout & 15 s

Table 3: Key System Parameters

7.1 Current Limitations

- **Simulated Weather:** The system currently does not ingest live weather API data; it relies on a sophisticated probability generator.
- **2D Movement:** Drones currently operate at a fixed altitude. True 3D terrain following is not yet implemented.
- **Simplified Physics:** Battery consumption is modeled as a linear drain, ignoring wind resistance and varied payload mass.

7.2 Research Modules Implemented

- **Swarm Ranging Protocol** (INFOCOM 2021): Implemented decentralized UWB-based localization. The `swarm_localization` node uses multilateration to estimate relative positions without relying on global GPS.
- **Energy-Aware Path Planning** (ICRA 2024): Integrated a battery discharge model and a mobile charging station (`ugv_manager`). The `energy_planner` coordinates rendezvous events for drones with critical battery levels ($<30\%$).

8 Implementation FAQs

Q: How was the INFOCOM 2021 Swarm Ranging paper implemented?

A: We simulated the hardware layer by creating a `uwb_simulator` node that calculates ground-truth distances and adds Gaussian noise to mimic real Ultra-Wideband sensors. A decentralized `swarm_localization` node then consumes these noisy ranges and uses a Multilateration (Least Squares) algorithm to estimate relative positions. This validates the paper’s core proposition of achieving relative localization without global GPS.

Q: How was the ICRA 2024 Energy Planning paper implemented?

A: We implemented:

1. **Battery Simulation:** An 800mAh capacity model with random initial charge (40-90%) to simulate realistic, staggered discharge profiles.
2. **Active Interception:** The UGV actively tracks and moves towards drones with critical battery levels ($< 30\%$).
3. **Charging Protocol:** A dedicated `/charge_cmd` topic ensures robust charging states, resetting batteries to 100% upon successful rendezvous.

Q: What inferences were made from the research papers?

A: We inferred that for a ROS-based simulation, the *behavioral logic* (rendezvous, range-based fixing) was more critical than exact physical modelling. Thus, we used linear approximations for battery drain and UWB noise models. We also adapted the centralized optimization algorithms from the papers into reactive ROS nodes to suit real-time simulation constraints.

8.1 Future Roadmap

1. **Live Weather APIs:** Replace the CSV data loader with real-time fetchers for OpenWeatherMap/MODIS APIs.
2. **Hardware Deployment:** Port the ROS nodes to a swarm of Bitcraze Crazyflie 2.1 drones for real-world field testing.

9 Recent Changes & Bug Fixes

This section documents all significant code changes made during the final development sprint (March – April 2026), covering four commits and one unversioned session.

9.1 Session: 18 April 2026 — Realistic Swarm Deployment, Portable LSTM Inference, and Clean Termination

Files modified: `launch/explore_areas.launch`, `scripts/area_explorer.py`, `scripts/drought_probabil`

Feature Enhancement: Portable LSTM Inference (Repository Lightweighting)

Requirement: The 2GB Kaggle Meteorological baseline dataset prevented the repository from being pushed smoothly to external Git hosts. **Implementation:**

1. Extracted a statistically valid 90-day sequential slice of the six required meteorological features from the CSV.
2. Saved this slice permanently within the repo as `LSTM/reference_sequence.npy` (~ 2.9 KB).
3. Updated `drought_probability_model.py` to attempt CSV load first. If the CSV directory is absent, it seamlessly catches the `FileNotFoundError` and routes the PyTorch tensor loader through `predict_from_reference_sequence()` instead. The inference network and random seed generator remain fully functional globally without relying on massive local hardware dependencies.

Feature Enhancement: Realistic Peer-to-Peer Crash Logging & Delayed Deployment

Root cause. On simulated hardware failure, the exploration loop incorrectly bypassed the Central Tower’s formal heartbeat timeout by forcefully injecting a `DEPLOY_RESERVE` command. This compromised realism by deploying a backup before the tower technically knew a drone was unresponsive.

Implementation.

1. Removed the eager `DEPLOY_RESERVE` intercept from the failure triggers.
2. Implemented simulated peer-to-peer (P2P) emergency mesh networking, where failing drones broadcast an ad-hoc SOS signal to immediate neighbors just before descent. Local drones use this signal to dynamically absorb the dead drone’s coverage sectors instantly.
3. Centralized backup dispatches now strictly obey the Central Tower’s independent heartbeat polling loop. Realistically, it takes the network a full 15-second timeout window to realize the drone is missing before a reserve launches.

Bug Fix: Simulation Nodes Lingering Post-Mission

Root cause. Though the `area_explorer.py` correctly fired a `rospy.signal_shutdown()` at the end of the simulation, other detached ROS nodes, background communication threads, and Gazebo did not self-terminate, leaving the program running indefinitely.

Fix. Added the `required="true"` attribute to the master `area_explorer` node inside `explore_areas.launch`. Now, when the primary mission completes and exits, the `roslaunch` process explicitly intercepts the termination and elegantly tears down the entire simulation environment.

9.2 Session: 5 April 2026 — Reserve Drone Activation & Stop-Condition Fixes

Files modified: `scripts/area_explorer.py`, `scripts/algo3_sim.py`

Bug Fix: Second Reserve Drone Remained Frozen After Deployment

Root cause. When two active drones died in the same 10-second heartbeat window, the Central Tower’s `broadcast_hello()` detected both timeouts in the same iteration of its `for dead_id in dead_agents` loop and published two consecutive `DEPLOY_RESERVE` commands to `/central/commands`.

Each backup was correctly popped from the `backups` list and promoted to a new `DroneExplorer`. However, the original `BackupDrone.hold_position()` thread running in the background had no exit condition. Its `while not rospy.is_shutdown()` loop remained alive and continued publishing `Twist()` (zero-velocity) commands to `/drone_X/cmd_vel`, directly fighting against the new explorer’s navigation commands. The second reserve consistently lost this race.

Fix.

1. Added `self.deployed = False` flag to `BackupDrone.__init__`.
2. Changed the `hold_position()` loop guard to: `while not rospy.is_shutdown() and not self.deployed`.
3. The `DEPLOY_RESERVE` handler now sets `bk.deployed = True` before constructing the `DroneExplorer`, guaranteeing the background thread exits within one 5 Hz tick.

Secondary fix. The promoted reserve’s initial position in `Algo3Hybrid` was hardcoded to (0, 0) (the field corner in local coordinates), causing incorrect Voronoi partitioning until the first real odometry arrived. It is now seeded from the backup drone’s actual current pose, transformed into local field coordinates.

Bug Fix: Simulation Continued Indefinitely After 100% Coverage

Root cause. `Algo3Hybrid.step()` contained a “continuous surveillance loop” that **reset** the entire coverage grid whenever `pct ≥ 99.9%`:

```
1 if current_cov_pct >= 99.9:
2     self.manager.visited = [False] * self.manager.total_points
3     self.manager.visited_by = [-1] * self.manager.total_points
4     self.best_coverage = 0.0
5     return "Surveillance Loop Reset"
```

This prevented `AreaCoverageController.is_complete()` (which checks `pct ≥ 100.0`) from ever returning `True`. Additionally, the main loop in `area_explorer.py` contained two identical `if active_areas == 0: stop` blocks, with unreachable dead code between them. The `mission_pub.publish()` call was also deferred until after the `while` loop, meaning UGVs and the Central Tower only stopped 5 seconds after the simulation had already ended.

Fix.

1. Replaced the grid-reset block with a simple early return: `return "Coverage Complete"`.
2. Merged the duplicate stop blocks into a single check: `if total_pct >= 100.0 or active_areas == 0:`
3. Moved `mission_pub.publish(Bool(data=True))` inside the `if` block so UGVs receive the stop signal immediately.
4. Removed the dead/unreachable code after the original `break`.

9.3 Commit 69c7b75 — 3 April 2026: UGV Deadlocks, Crash Visuals, Dead-Drone Edge Cases

Files modified: `models/dead_quadcopter/model.sdf`, `scripts/algo3_sim.py`, `scripts/area_explorer.py`, `scripts/energy_planner.py`, `scripts/ugv_manager.py`

- **UGV deadlock resolution:** Removed the redundant local intercept calculation from `ugv_manager.py`. Both UGVs were independently computing intercept vectors towards the same target drone, causing them to oscillate into a deadlock. The UGV now follows the path computed exclusively by `energy_planner.py`.
- **Crash teleport:** On hardware failure, the dead drone is teleported to $z = -100\text{ m}$ via the Gazebo `SetModelState` service. This clears the physics engine's model handle before the static `dead_quadcopter` SDF is spawned, eliminating mutex-locking errors that previously froze the simulation.
- **Ghost cone deletion:** A `Marker` message with `action = DELETE` for the dead drone's FOV cone is published to `/drone_{id}/cone_marker` immediately before the teleport, removing phantom cyan cones in RViz.
- **Dead-drone battery sentinel:** Upon simulated hardware failure, the area explorer immediately publishes `Float32(data=100.0)` to `/drone_{id}/battery`. This prevents UGV energy planners from treating the dead drone as a valid charging candidate.

9.4 Commit f6431ef — 2 April 2026: Fleet Stagnation & Dynamic Reconfiguration

Files modified: `config/areas.yaml`, `scripts/algo3_sim.py`, `scripts/area_explorer.py`, `scripts/central_agent.py`, `scripts/drone_comm.py`, `scripts/spawn_fleet.py`, `worlds/field_areas.world`

- **waiting_for_charge stall:** Drones returning to base for recharging stalled indefinitely in the `waiting_for_charge` state. Fixed by clearing the flag as soon as `battery.get_percentage() > 90.0` inside the `charge_callback`, allowing immediate mission resumption.
- **Dead-drone heartbeat silencing:** `drone_comm.py` now subscribes to `/swarm/drone_death`. On receiving a dead drone's global ID, it sets an internal `is_dead` flag and stops responding to any `HELLO` or `HELLO_RETRY` messages, preventing the Central Tower from wasting retry budget on wrecks.
- **Central Tower retry hardening:** Added targeted 2.2-second unicast retries (`HELLO_RETRY_{<ID>}`) after each broadcast round. Agents missing two consecutive heartbeat windows ($>15\text{ s}$) are classified as dead and a single `DEPLOY_RESERVE` command is published per dead agent.
- **Unified workspace:** The five scattered 10–12 m circular farmland areas were merged into a single **21.5 m × 21.5 m** rectangular `unified_workspace` in both `field_areas.world` and `areas.yaml`. All active drones now cover this shared workspace under dynamic Voronoi partitioning, eliminating inter-area boundary conflicts and partition starvation.
- **Reserve-aware spawning** (`spawn_fleet.py`): Reads the `reserve_drones` parameter from `areas.yaml` at startup. Explorer drones are spawned in an inner ring; reserve drones are placed in a wider outer ring to avoid formation collisions.

9.5 Commit 8ed9f43 — 13 March 2026: Scanning State Machine Fix

Files modified: `scripts/algo3_sim.py`, `scripts/area_explorer.py`

- **Premature waypoint clearance:** The previous implementation marked a waypoint as visited the same tick the drone arrived, even if a scanning pause had not yet elapsed. This allowed the algorithm to immediately re-assign the drone to the next target, causing phantom coverage inflation and a “re-visit loop” where the same points were scanned multiple times.
- **Fix:** Introduced a `just_finished` boolean in `DroneExplorer`. The scanning FSM now proceeds: *arrive* → `start_scanning()` → hold-hover for 1 s → `check_just_finished()` → `force_mark_visited()` → request new target. The Algo update tick is skipped during the scan pause, ensuring the waypoint is only marked once and exactly at completion.